

Taking a leap of faith in recursion is not a magical technique that guarantees your code works bug free. It is merely a perspective to hold to break past the mental programmer's block you can have when thinking about how to implement your recursive function. The leap of faith requires you to have a firm understanding of your recursive function's arguments and return value.

Note that the leap-of-faith technique only helps you write the recursive case. You must pass to the recursive call an argument that is closer to the base case. You can't simply pass the same argument that the recursive function received, like this:

```
def rev(theString):  
    return rev(theString) # This won't magically work.
```

To continue our 'CAT' example, when we pass the tail 'AT' to `rev()`, the head is 'A' and the tail is 'T' in *that* function call. We already know that the reverse of a single-character string like 'T' is simply 'T'; that's our base case. So this second call to `rev()` will reverse 'AT' to 'TA', which is precisely what the previous call to `rev()` needs. [Figure 3-3](#) shows the state of the call stack during all the recursive calls to `rev()`.

Let's ask our three recursive algorithm questions about the `rev()` function:

What is the base case? A zero- or one-character string.

What argument is passed to the recursive function call? The tail of the original string argument, which has one less character than the original string argument.

How does this argument become closer to the base case? The array argument shrinks by one element for each recursive call until it becomes a one- or zero-length array.